

Business Requirements: Cardano RWA Instant Liquidity Pool

1. Project Overview

Build a Cardano-based RWA liquidity pool using Aiken smart contracts.

The system allows users to sell approved real-world asset tokens, such as tokenized sealed Pokémon packs, to a shared liquidity pool for instant settlement. The pool pays sellers using a settlement asset such as ADA, USDM, DJED, or another Cardano native asset.

Liquidity providers deposit settlement assets into the pool and earn from fees, bid/ask spreads, and pool value growth. Buyers may also buy RWA tokens from the pool. The pool should keep RWA inventory on-chain under the pool validator unless an operator settlement flow explicitly removes inventory in exchange for settlement value.

The system must use oracle-controlled pricing and registry-controlled asset permissions.

2. Main Goals

The contracts should support:

1. Users selling approved RWA tokens to the pool for instant liquidity.
 2. Buyers purchasing RWA tokens from the pool.
 3. Liquidity providers depositing settlement assets and receiving LP share tokens.
 4. Liquidity providers withdrawing liquidity by burning LP share tokens.
 5. Oracle operators publishing bid, ask, and NAV/settlement prices.
 6. Governance/admin operators controlling asset permissions, risk parameters, fees, and pause settings.
 7. Shared liquidity across many RWA asset buckets while enforcing per-asset and per-policy exposure controls.
 8. Policy-level default pricing with optional asset-level price overrides.
 9. Asset registry logic for allowed policies, allowed assets, risk tiers, enabled/disabled flags, and exposure caps.
 10. Pokémon pack-style RWA tokens where one token may represent one physical sealed pack, box, bundle, or other redeemable item.
-

3. Key Actors

Seller

A seller holds an approved RWA token and sells it to the pool.

The seller receives the settlement asset based on the oracle bid price, less applicable fees.

Buyer

A buyer pays the settlement asset to purchase RWA tokens from the pool.

The buyer pays based on the oracle ask price plus applicable fees.

Liquidity Provider

A liquidity provider deposits the settlement asset into the liquidity vault/pool.

In return, the LP receives LP share tokens representing economic ownership of the pool.

LPs earn through:

- Buy/sell spread.
- Trading fees.
- Protocol-defined LP fee share.
- Increase in pool NAV from profitable inventory turnover.

Pool Operator / Governance

The operator or governance authority manages:

- Allowed asset policies and asset buckets.
- Risk parameters.
- Fee parameters.
- Oracle operators.
- Pool pause/unpause status.
- Asset freeze/default status.
- Exposure caps.
- Contract migration or upgrade controls, if supported.

Oracle Operator

The oracle operator publishes current pricing data used by the pool validator.

The oracle should publish:

- Bid price: price paid by the pool when buying from sellers.

- Ask price: price paid by buyers when buying from the pool.
- NAV/settlement price: price used for LP valuation or operator inventory settlement.

Custodian

The custodian stores and verifies the physical RWA items, such as sealed Pokémon packs.

The custodian handles:

- Inspection.
- Authentication.
- Vault storage.
- Redemption.
- Shipping or pickup.

Redeemer

A redeemer burns or submits an RWA token to claim the corresponding physical asset through the custodian.

Redemption may be handled off-chain, but the on-chain registry should support redeem-enabled/redeemed/frozen status.

4. Contract Components

The system should be split into multiple contracts/policies:

Pool Contract

The main validator controlling the pool UTxO.

Responsibilities:

- Holds settlement liquidity.
- Holds RWA inventory.
- Enforces sell-to-pool transactions.
- Enforces buy-from-pool transactions.
- Enforces deposit/withdraw liquidity rules.
- Enforces oracle price usage.
- Enforces registry asset permissions.
- Enforces fee collection.
- Enforces exposure limits.
- Enforces pause state.

LP Share Minting Policy

Mints and burns LP share tokens.

Responsibilities:

- Mint LP shares when settlement liquidity is deposited.
- Burn LP shares when liquidity is withdrawn.
- Only allow mint/burn when the pool validator is also involved.
- Prevent arbitrary LP token minting.

Oracle Contract

Controls the oracle UTxO containing pricing data.

Responsibilities:

- Store oracle NFT.
- Store current price datum.
- Allow authorized oracle updates.
- Enforce sequence increase.
- Enforce validity window.
- Enforce basic price sanity checks.

Registry Contract

Controls allowed asset and policy configuration.

Responsibilities:

- Store registry NFT.
- Store allowed policies.
- Store optional asset-level overrides.
- Track active/frozen/defaulted/redeemed flags.
- Track sell-enabled, buy-enabled, and redeem-enabled flags.
- Store max trade and max exposure parameters.
- Allow authorized governance/admin updates.

Governance / Admin Control

This may be implemented using an admin NFT, multisig policy, or governance validator.

Responsibilities:

- Authorize registry updates.
- Authorize oracle operator changes.
- Authorize pool parameter updates.

- Pause/unpause pool.
- Freeze/unfreeze assets or policies.
- Manage protocol fee settings.
- Manage contract migration, if needed.

One-Shot NFT Minting Policy

Mint unique NFTs for:

- Pool NFT.
- Oracle NFT.
- Registry NFT.
- Admin NFT.

These NFTs identify authentic state UTxOs.

5. Core Tokens

RWA Token

Represents a claim on a real-world asset.

For Pokémon packs:

- One token may represent one physical sealed pack.
- One token may represent one sealed box, bundle, or case.
- One token may represent a fractional claim on a vault lot, if explicitly supported.

Recommended approach:

- One token per physical item for redeemable packs/boxes.
- Policy per product bucket.
- Asset name as unique unit ID.

Example product bucket:

```
series + set + language + product type + condition class
```

Example:

```
Scarlet & Violet / Journey Together / English / Sleeved Booster Pack / Sealed
```

LP Share Token

Represents economic ownership of the shared liquidity pool.

LP tokens entitle holders to withdraw a proportional share according to the contract's withdrawal logic.

Pool NFT

Identifies the authentic pool UTxO.

The pool validator must require that the pool NFT remains locked at the pool script address.

Oracle NFT

Identifies the authentic oracle UTxO.

The pool validator should only trust a reference input containing the oracle NFT.

Registry NFT

Identifies the authentic asset registry UTxO.

The pool validator should only trust a reference input containing the registry NFT.

Admin NFT

Represents administrative control, or may be locked in a multisig/governance contract.

6. Pool Business Logic

Sell RWA to Pool

A seller can sell an approved RWA token to the pool.

Inputs:

- Seller's RWA token.
- Pool UTxO.
- Oracle reference input.
- Registry reference input.

Outputs:

- Seller receives settlement asset.
- Pool receives RWA token.

- Pool settlement reserve decreases.
- Pool UTxO is recreated with pool NFT and updated value.

Validator must check:

1. Pool is not paused.
2. Asset policy is allowed by registry.
3. Asset is active and not frozen/defaulted/redeemed.
4. Sell-to-pool is enabled.
5. Oracle price is valid and not expired.
6. Price is resolved using asset override first, then policy default.
7. Seller receives at least required payout.
8. Pool receives the RWA amount.
9. Pool settlement reserve decreases by the correct amount.
10. Trade does not exceed max trade limits.
11. Asset exposure does not exceed max exposure.
12. Pool cash reserve remains above minimum cash requirement.
13. Pool NFT remains at the pool script.

Formula:

```
gross_value = rwa_amount * bid_price

seller_payout =
    gross_value * (10_000 - total_fee_bps) / 10_000
```

Optional protocol fee:

```
protocol_fee =
    gross_value * protocol_fee_bps / 10_000
```

LP fee remains inside the pool.

Buy RWA from Pool

A buyer can buy RWA inventory from the pool.

Inputs:

- Buyer settlement asset.
- Pool UTxO.
- Oracle reference input.
- Registry reference input.

Outputs:

- Buyer receives RWA token.
- Pool receives settlement asset.
- Pool RWA inventory decreases.
- Pool UTxO is recreated.

Validator must check:

1. Pool is not paused.
2. Asset is allowed.
3. Asset is active and buy-from-pool is enabled.
4. Oracle ask price is valid and not expired.
5. Buyer pays at least required ask price.
6. Buyer receives no more RWA than paid for.
7. Pool loses exactly the sold RWA amount.
8. Pool settlement reserve increases by required payment.
9. Pool NFT remains at the pool script.

Formula:

$$\text{buyer_payment} = \text{rwa_amount} * \text{ask_price}$$

Provide Liquidity

An LP deposits settlement asset into the pool and receives LP share tokens.

Inputs:

- LP settlement asset.
- Pool UTxO.
- Optional NAV oracle reference input.
- LP minting policy.

Outputs:

- Pool receives settlement asset.
- LP receives LP share tokens.
- Pool datum updates total LP supply.

Validator must check:

1. Deposit asset is the pool settlement asset.
2. LP tokens are minted according to the pool share formula.
3. Pool settlement reserve increases by the deposit amount.

4. LP receiver receives the minted LP tokens.
5. Pool NFT remains at the pool script.
6. Total LP supply is updated.

For initial deposit:

```
lp_to_mint = deposit_amount
```

For later deposits, recommended practical approach:

```
lp_to_mint = deposit_amount * total_lp_supply / pool_nav
```

The pool NAV may come from a NAV oracle.

Withdraw Liquidity

An LP burns LP share tokens and receives settlement asset.

Inputs:

- LP share tokens.
- Pool UTxO.
- Optional NAV oracle reference input.
- LP minting policy.

Outputs:

- LP receives settlement asset.
- LP tokens are burned.
- Pool reserve decreases.
- Pool datum updates total LP supply.

Validator must check:

1. LP tokens are burned.
2. Withdrawal amount is calculated correctly.
3. Receiver gets at least minimum requested settlement output.
4. Pool remains above minimum cash reserve if required.
5. Pool NFT remains at the pool script.
6. Total LP supply decreases correctly.

Simple formula:

```
withdraw_amount =  
  lp_amount * withdrawable_liquidity / total_lp_supply
```

If using NAV-based withdrawals:

```
withdraw_amount =  
  lp_amount * pool_nav / total_lp_supply
```

However, withdrawals must still be capped by available settlement liquidity.

Operator Inventory Settlement

The operator may remove RWA inventory from the pool only by depositing equivalent settlement value.

This allows the operator to redeem or manage physical assets off-chain without stealing LP-owned inventory.

Inputs:

- Pool UTxO.
- Operator settlement payment.
- Oracle reference input.
- Registry reference input.
- Operator/admin authorization.

Outputs:

- Operator receives RWA token.
- Pool receives settlement asset.
- Pool RWA inventory decreases.
- Pool UTxO is recreated.

Validator must check:

1. Operator/admin is authorized.
2. Oracle settlement price is valid.
3. Pool receives at least settlement value.
4. Operator receives only the specified RWA amount.
5. Pool NFT remains at the pool script.

Formula:

```
settlement_payment =  
    rwa_amount * settlement_price
```

7. Pricing Requirements

Pricing must be oracle-controlled, not AMM-discovered.

The oracle should support:

1. Policy-level default pricing.
2. Asset-level price overrides.
3. Bid price.
4. Ask price.
5. NAV/settlement price.
6. Expiry/validity timestamp.
7. Sequence number.
8. Active/inactive flag.

Price Resolution Rule

When pricing an asset:

1. Look for asset-specific override by policy_id + asset_name.
2. If found and active, use it.
3. Otherwise, look for policy-level default price.
4. If found and active, use it.
5. Otherwise, reject the transaction.

Price Sanity Rules

The oracle contract should reject invalid updates where:

1. Denominator is zero or negative.
2. Bid price is zero or negative.
3. Ask price is zero or negative.
4. Bid is greater than ask.
5. NAV/settlement price is outside allowed sanity bounds, if enforced.
6. Sequence number does not increase.
7. Validity timestamp is not in the future.

Recommended invariant:

```
bid <= nav <= ask
```

8. Shared Liquidity Requirements

The pool should share one settlement reserve across many allowed RWA assets.

This allows idle liquidity to be used across different asset buckets.

However, shared liquidity must be risk-controlled.

The registry should support:

1. Per-policy exposure cap.
2. Per-asset exposure cap.
3. Per-trade maximum.
4. Global RWA exposure cap.
5. Minimum cash reserve.
6. Risk tier.
7. Sell-enabled flag.
8. Buy-enabled flag.
9. Redeem-enabled flag.
10. Frozen/defaulted/redeemed flags.

Validator must ensure:

```
new_asset_exposure <= asset_exposure_cap  
new_policy_exposure <= policy_exposure_cap  
new_total_rwa_exposure <= global_exposure_cap  
pool_cash_after >= min_cash_reserve
```

9. Asset Registry Requirements

The registry must support both policy-level and asset-level controls.

Policy Config

Each policy config should include:

- policy_id
- active
- sell_to_pool_enabled

- buy_from_pool_enabled
- redeem_enabled
- max_policy_exposure_value
- max_trade_value
- risk_tier
- requires_kyc
- product bucket metadata hash or bucket ID

Asset Config Override

Each asset-level config may include:

- asset_policy_id
- asset_name
- active
- frozen
- defaulted
- redeemed
- sell_to_pool_enabled override
- buy_from_pool_enabled override
- redeem_enabled override
- max_asset_exposure_value
- max_trade_value
- requires_kyc override
- condition override
- reason code

Registry Resolution Rule

1. Policy must exist and be active.
2. Asset override may apply stricter controls.
3. Asset override should not bypass a disabled policy.
4. Effective max trade should be the stricter of policy and asset settings.
5. Effective KYC requirement should be policy OR asset requirement.

10. Pokémon Pack-Specific Business Rules

For Pokémon packs, the recommended business model is:

Policy per product bucket.
Asset per physical item.

A product bucket should represent:

series + set + language + product type + condition class

Examples:

SV_JOURNEY_TOGETHER_EN_SLEEVED_PACK
SV_PRISMATIC_EVOLUTIONS_EN_BOOSTER_BUNDLE
SWSH_SILVER_TEMPEST_EN_BOOSTER_PACK
VINTAGE_BASE_SET_EN_PACK_AUTHENTICATED

Each physical item should have a unique token asset name:

PACK_000001
PACK_000002
BOX_000001

Policy-level flat pricing is allowed only when all assets under the policy represent the same product type and condition.

Asset-level override pricing is required for:

- Damaged packaging.
- Vintage packs.
- Weighed packs.
- Special provenance.
- Different language.
- Different region.
- Different product format.
- Suspicious or frozen items.
- Individually appraised items.

11. Custody and Redemption Requirements

The system must distinguish between token ownership and physical custody.

A token should be sellable only if:

1. The physical item has been received.
2. The item has been inspected.
3. The item is verified as authentic/sealed.
4. The item is stored in custody.
5. The registry marks the item or bucket as active.

Redemption should mark the item/token as redeemed or prevent it from being sold again.

Redemption may be implemented as:

1. On-chain burn or lock of RWA token.
2. Off-chain custodian fulfillment.
3. Registry update to mark redeemed status.

12. Fee Model

The pool may charge fees on both buy and sell transactions.

Fee types:

1. LP fee.
2. Protocol fee.
3. Custody/redemption fee, if applicable.
4. Optional operator settlement fee.

LP fees should generally remain inside the pool, increasing LP share value.

Protocol fees may be paid to a treasury output.

Example:

```
seller_payout =  
gross_bid_value - lp_fee - protocol_fee
```

For buys:

```
buyer_payment =  
gross_ask_value + protocol_fee, if separate
```

The validator must enforce treasury payment if protocol fees are enabled.

13. Required Pool Parameters

The pool datum should include:

- pool_nft
- lp_token

- settlement_asset
 - oracle_nft
 - registry_nft
 - admin_nft or admin credential
 - total_lp_supply
 - fee_bps
 - protocol_fee_bps
 - min_cash_reserve
 - max_total_rwa_exposure
 - treasury address or credential
 - paused flag
-

14. Required Oracle Parameters

The oracle datum should include:

- oracle_nft
- policy_id or bucket_id
- sequence
- valid_until
- default policy quote
- optional asset-level overrides

Quote should include:

- bid ratio
 - ask ratio
 - nav/settlement ratio
 - active flag
 - max trade value, optional
-

15. Required Registry Parameters

The registry datum should include:

- registry_nft
- sequence
- policy configs
- asset configs / overrides

Policy configs should include:

- policy_id
- bucket_id
- active flag

- sell enabled
- buy enabled
- redeem enabled
- max exposure
- max trade
- risk tier
- requires KYC

Asset configs should include:

- policy_id
 - asset_name
 - frozen/defaulted/redeemed flags
 - optional overrides
 - max exposure
 - max trade
 - reason code
-

16. Initial Setup Flow

The protocol setup should follow this flow:

1. Choose settlement asset.
 2. Define initial fees, exposure limits, and admin credentials.
 3. Mint pool NFT, oracle NFT, registry NFT, and admin NFT.
 4. Deploy pool validator.
 5. Deploy oracle validator.
 6. Deploy registry validator.
 7. Deploy LP minting policy.
 8. Create registry UTxO with allowed policies/assets.
 9. Create oracle UTxO with initial prices.
 10. Create pool UTxO with pool NFT and initial settlement liquidity.
 11. Mint initial LP share tokens to the initial liquidity provider.
 12. Start indexing pool, oracle, and registry state.
-

17. Main Invariants

The contracts must preserve these invariants:

1. Pool NFT must always remain locked at the pool validator.
2. Oracle reference input must contain the oracle NFT.
3. Registry reference input must contain the registry NFT.
4. Pool cannot trade paused, frozen, defaulted, or unapproved assets.
5. Seller payout must match oracle bid pricing.

6. Buyer payment must match oracle ask pricing.
 7. Operator cannot remove RWA inventory without depositing required settlement value.
 8. LP shares cannot be minted except through valid liquidity deposits.
 9. LP shares cannot be burned except through valid withdrawals.
 10. Pool must remain above minimum cash reserve, if configured.
 11. Exposure caps must not be exceeded.
 12. Protocol fee must be paid to treasury when enabled.
 13. Oracle prices must not be expired.
 14. Oracle sequence must increase on update.
 15. Registry sequence must increase on update.
-

18. MVP Scope

For the first version, implement:

1. One settlement asset.
2. One shared liquidity pool.
3. One pool NFT.
4. One LP share token.
5. Registry with policy-level controls.
6. Optional asset-level registry overrides.
7. Oracle with policy-level default pricing.
8. Optional asset-level price overrides.
9. Sell RWA to pool.
10. Buy RWA from pool.
11. Deposit liquidity.
12. Withdraw liquidity.
13. Admin pause/unpause.
14. Registry update.
15. Oracle price update.
16. Operator inventory settlement.

Defer for later:

1. Full DAO governance.
 2. Complex time-window throttling.
 3. Merkle proof-based pricing.
 4. Cross-chain settlement.
 5. Automated redemption fulfillment.
 6. Advanced NAV computation fully on-chain.
 7. Secondary lending or collateralization.
-

19. Recommended Contract Files

Suggested Aiken modules:

```
validators/pool.ak  
validators/oracle.ak  
validators/registry.ak  
validators/lp_token.ak  
validators/one_shot_nft.ak
```

```
lib/types.ak  
lib/value.ak  
lib/math.ak  
lib/oracle.ak  
lib/registry.ak  
lib/pool_helpers.ak
```

20. Acceptance Criteria

The implementation should include tests proving:

1. Valid seller transaction succeeds.
2. Seller transaction fails if oracle is missing.
3. Seller transaction fails if price is expired.
4. Seller transaction fails if asset is not allowed.
5. Seller transaction fails if payout is too low.
6. Seller transaction fails if pool does not receive RWA.
7. Buyer transaction succeeds.
8. Buyer transaction fails if payment is too low.
9. Buyer transaction fails if pool does not send RWA.
10. LP deposit succeeds and mints correct LP shares.
11. LP withdrawal succeeds and burns correct LP shares.
12. LP minting fails outside valid deposit flow.
13. LP burning fails outside valid withdrawal flow.
14. Oracle update succeeds with authorized signer/admin.
15. Oracle update fails with unauthorized signer.
16. Registry update succeeds with authorized admin.
17. Registry update fails with unauthorized admin.
18. Paused pool rejects buy/sell.
19. Frozen asset rejects buy/sell.
20. Asset-level price override takes priority over policy default.
21. Policy-level price is used when no asset override exists.
22. Exposure cap violations are rejected.
23. Minimum cash reserve violations are rejected.

- 24. Operator settlement succeeds only when settlement value is deposited.
- 25. Operator cannot withdraw RWA inventory for free.